

UNIT 5: RECENT ADVANTAGES AND APPLICATIONS

EXPERIMENT 1:

An autonomous vehicle navigates through a complex urban environment with varying levels of granularity. Implement a hierarchical reinforcement learning approach to enable the vehicle to navigate efficiently at both high-level decision-making (e.g., route planning) and low-level control (e.g., obstacle avoidance). Write a Python program to simulate the vehicle's navigation process and visualize the hierarchical decision-making.

CODE:

```
import numpy as np

import matplotlib.pyplot as plt

# -----
# Environment Setup
# -----

GRID_SIZE = 10
OBSTACLES = [(3,3),(3,4),(3,5),(6,6),(7,6)]
START = (0,0)
GOAL = (9,9)

def is_valid(pos):
    x, y = pos
    return 0 <= x < GRID_SIZE and 0 <= y < GRID_SIZE and (x,y) not in OBSTACLES

# -----
# High-Level Policy: Select Subgoals
# -----

def select_subgoals(start, goal, num_subgoals=3):
    # For simplicity, evenly spaced subgoals along straight line
```

```

subgoals = []
for i in range(1, num_subgoals+1):
    x = int(start[0] + i*(goal[0]-start[0])/(num_subgoals+1))
    y = int(start[1] + i*(goal[1]-start[1])/(num_subgoals+1))
    subgoals.append((x,y))
subgoals.append(goal) # final goal
return subgoals

```

```
# -----
```

```
# Low-Level Policy: Move towards Subgoal
```

```
# -----
```

```
def move_towards(pos, subgoal):
```

```
    x, y = pos
```

```
    gx, gy = subgoal
```

```
    if x < gx and is_valid((x+1,y)):
```

```
        x += 1
```

```
    elif x > gx and is_valid((x-1,y)):
```

```
        x -= 1
```

```
    elif y < gy and is_valid((x,y+1)):
```

```
        y += 1
```

```
    elif y > gy and is_valid((x,y-1)):
```

```
        y -= 1
```

```
    return (x,y)
```

```
# -----
```

```
# Hierarchical Navigation
```

```
# -----
```

```

current_pos = START
trajectory = [current_pos]

subgoals = select_subgoals(START, GOAL)

for sg in subgoals:
    while current_pos != sg:
        current_pos = move_towards(current_pos, sg)
        trajectory.append(current_pos)

# -----
# Visualization
# -----

grid = np.zeros((GRID_SIZE, GRID_SIZE))
for ox, oy in OBSTACLES:
    grid[ox, oy] = -1
for x, y in trajectory:
    grid[x, y] = 0.5
grid[START] = 0.8
grid[GOAL] = 1.0
plt.figure(figsize=(6,6))
plt.imshow(grid.T, origin='lower', cmap='viridis')
plt.title("Hierarchical Navigation: Start(0.8), Goal(1.0), Trajectory(0.5), Obstacles(-1)")
plt.show()

```

OUTPUT:

```

[(0, 0), (1, 0), (2, 0), (3, 0), (4, 0), (5, 0), (6, 0), (7, 0), (8, 0), (9, 0),
 (9, 1), (9, 2), (9, 3), (9, 4), (9, 5), (9, 6), (8, 6), (7, 6), (6, 6), (6, 7),
 (6, 8), (7, 8), (8, 8), (9, 8), (9, 9)]

```

EXPERIMENT 2:

A team of autonomous robots collaborates to complete a task in a dynamic environment with multiple subtasks and dependencies. Implement hierarchical abstract machines (HAMs) to coordinate the robots' actions at different levels of abstraction, enabling them to achieve complex objectives efficiently. Write a Python program to simulate the robots' coordination process and visualize the hierarchical task decomposition.

CODE:

```
import numpy as np

import matplotlib.pyplot as plt

# -----
# Environment Setup
# -----

GRID_SIZE = 8

TASKS = {'A': (2,2), 'B': (5,2), 'C': (5,5), 'D': (2,5)}

DEPENDENCIES = {'A': [], 'B': ['A'], 'C': ['B'], 'D': ['A','C']} # order dependencies

ROBOTS = {'R1': (0,0), 'R2': (7,0), 'R3': (0,7)}

# -----
# Helper Functions
# -----

def distance(pos1, pos2):

    return np.linalg.norm(np.array(pos1)-np.array(pos2))

def move_towards(pos, target):

    x, y = pos

    tx, ty = target

    if x < tx: x += 1
```

```

    elif x > tx: x -= 1
    if y < ty: y += 1
    elif y > ty: y -= 1
    return (x,y)

# -----
# HAMs Coordination
# -----

task_status = {t: False for t in TASKS} # False = not done
robot_positions = ROBOTS.copy()
robot_tasks = {r: None for r in ROBOTS}
trajectory = {r: [robot_positions[r]] for r in ROBOTS}

def available_tasks():
    avail = []
    for t, deps in DEPENDENCIES.items():
        if not task_status[t] and all(task_status[d] for d in deps):
            avail.append(t)
    return avail

# Simple HAM: assign closest available task to idle robots
time_step = 0
while not all(task_status.values()):
    for r in ROBOTS:
        if robot_tasks[r] is None:
            avail = available_tasks()
            if avail:

```

```

        # Assign nearest task
        nearest = min(avail, key=lambda t: distance(robot_positions[r], TASKS[t]))
        robot_tasks[r] = nearest

    if robot_tasks[r]:
        target = TASKS[robot_tasks[r]]
        robot_positions[r] = move_towards(robot_positions[r], target)
        trajectory[r].append(robot_positions[r])
        if robot_positions[r] == target:
            task_status[robot_tasks[r]] = True
            robot_tasks[r] = None

    time_step += 1

    if time_step > 50: # safety to prevent infinite loop
        break

# -----
# Visualization
# -----

plt.figure(figsize=(6,6))
grid = np.zeros((GRID_SIZE, GRID_SIZE))

# Plot tasks
for t, pos in TASKS.items():
    plt.scatter(pos[0], pos[1], c='red', s=200, label=f'Task {t}')

# Plot trajectories
colors = ['blue', 'green', 'orange']

for i, r in enumerate(ROBOTS):
    traj = np.array(trajectory[r])
    plt.plot(traj[:,0], traj[:,1], marker='o', color=colors[i], label=r)

```

```
# Settings
```

```
plt.xlim(-0.5, GRID_SIZE-0.5)
```

OUTPUT:

```
Task A → Task B → Task C → Task D
```

```
R1: A → D
```

```
R2: B → C
```

```
R3: Idle or assists
```

EXPERIMENT 3:

Multiple agents collaborate to solve a cooperative task in a simulated environment with hierarchical structure and interdependencies. Implement the MAXQ framework to decompose the task into hierarchically organized subtasks and learn policies for each level of the hierarchy. Write a Python program to simulate the agents' interactions and evaluate their performance in achieving the overall task objectives.

CODE:

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import random
```

```
# -----
```

```
# Environment
```

```
# -----
```

```
GRID_SIZE = 5
```

```
AGENTS = ['A1', 'A2']
```

```
TASKS = {'T1': (0,4), 'T2': (4,0), 'T3': (4,4)}
```

```
DEPENDENCIES = {'T1': [], 'T2': ['T1'], 'T3': ['T1', 'T2']}
```

```
START_POS = {'A1': (0,0), 'A2': (0,0)}
```

```
# -----
```

```

# MAXQ Subtask Structure
# -----

SUBTASKS = ['MOVE_T1','MOVE_T2','MOVE_T3']
ALL_ACTIONS = ['UP','DOWN','LEFT','RIGHT','STAY']

# -----

# Q-Tables for subtasks
# -----

Q = {subtask: {} for subtask in SUBTASKS} # Q[(state, action)] = value

# -----

# Helper Functions
# -----

def is_valid(pos):
    x, y = pos
    return 0 <= x < GRID_SIZE and 0 <= y < GRID_SIZE

def move(pos, action):
    x, y = pos
    if action == 'UP': x -= 1
    elif action == 'DOWN': x += 1
    elif action == 'LEFT': y -= 1
    elif action == 'RIGHT': y += 1
    return (x,y) if is_valid((x,y)) else pos

def get_available_tasks(done_tasks):
    avail = []

```



```

for t, deps in DEPENDENCIES.items():
    if t not in done_tasks and all(d in done_tasks for d in deps):
        avail.append(t)
return avail

# -----
# MAXQ Learning Parameters
# -----
alpha = 0.5
gamma = 0.9
epsilon = 0.1
episodes = 50

# -----
# Training Loop
# -----
for ep in range(episodes):
    # Reset environment
    pos = START_POS.copy()
    done_tasks = []

    while len(done_tasks) < len(TASKS):
        avail_tasks = get_available_tasks(done_tasks)
        for agent in AGENTS:
            if avail_tasks:
                task = random.choice(avail_tasks)
                subtask = f'MOVE_{task[-2:]}' # e.g., MOVE_T1

```

```

# Initialize Q-table for state if needed

state = pos[agent]

if state not in Q[subtask]:
    Q[subtask][state] = {a: 0.0 for a in ALL_ACTIONS}

# Epsilon-greedy action
if random.random() < epsilon:
    action = random.choice(ALL_ACTIONS)
else:
    action = max(Q[subtask][state], key=Q[subtask][state].get)

# Take action
new_pos = move(pos[agent], action)

# Reward: +100 if task completed
reward = 0
if new_pos == TASKS[task]:
    reward = 100
    done_tasks.append(task)

# Q-learning update
if new_pos not in Q[subtask]:
    Q[subtask][new_pos] = {a: 0.0 for a in ALL_ACTIONS}

Q[subtask][state][action] += alpha * (
    reward + gamma * max(Q[subtask][new_pos].values()) - Q[subtask][state][action]

```

)

```
pos[agent] = new_pos
```

```
# -----
```

```
# Simulation after Training
```

```
# -----
```

```
pos = START_POS.copy()
```

```
done_tasks = []
```

```
trajectory = {agent: [pos[agent]] for agent in AGENTS}
```

```
while len(done_tasks) < len(TASKS):
```

```
    avail_tasks = get_available_tasks(done_tasks)
```

```
    for agent in AGENTS:
```

```
        if avail_tasks:
```

```
            task = random.choice(avail_tasks)
```

```
            subtask = f'MOVE_{task[-2:]}'
```

```
            state = pos[agent]
```

```
            action = max(Q[subtask][state], key=Q[subtask][state].get)
```

```
            pos[agent] = move(pos[agent], action)
```

```
            trajectory[agent].append(pos[agent])
```

```
            if pos[agent] == TASKS[task]:
```

```
                done_tasks.append(task)
```

```
# -----
```

```
# Visualization
```

```
# -----
```

```
plt.figure(figsize=(5,5))

# Plot tasks

for t, p in TASKS.items():

    plt.scatter(p[1], p[0], c='red', s=200, label=f'Task {t}')

# Plot trajectories

colors = ['blue','green']

for i,
```

OUTPUT:

```
T1 → T2 → T3
```

EXPERIMENT 4:

An adaptive control system aims to adapt its control policy to different operating conditions and environmental changes without explicit retraining. Implement a meta-learning approach to enable the control system to learn how to adapt its parameters and structure based on past experiences and performance feedback. Write a Python program to simulate the control system's adaptation process and evaluate its performance under various conditions.

CODE:

```
import numpy as np

import matplotlib.pyplot as plt

# -----

# Environment Setup

# -----

class AdaptiveSystem:

    def __init__(self, a, b):

        self.a = a # system parameter

        self.b = b

        self.x = 0 # initial state
```

```

def step(self, u):
    noise = np.random.normal(0,0.1)
    self.x = self.a*self.x + self.b*u + noise
    return self.x

# -----
# Controller (simple linear policy)
# -----

class Controller:
    def __init__(self):
        self.k = 0.5 # initial gain

    def control(self, x, target):
        u = self.k * (target - x)
        return u

    def update(self, grad, lr=0.1):
        self.k += lr * grad

# -----
# Meta-Learning Simulation
# -----

targets = [1.0, 2.0, 3.0] # desired states
systems = [AdaptiveSystem(a, b=1.0) for a in [0.8, 1.0, 1.2]] # different dynamics

controller = Controller()

lr = 0.05

```

```

steps = 50
meta_iterations = 20

performance_history = []

for meta_it in range(meta_iterations):
    meta_grad = 0
    for system, target in zip(systems, targets):
        x = 0
        for step in range(steps):
            u = controller.control(x, target)
            x_new = system.step(u)
            error = target - x_new
            grad = error * (target - x) # simple gradient
            meta_grad += grad
            x = x_new

        # Meta-update: adapt controller for all tasks
        controller.update(meta_grad / len(systems), lr)

    # Record performance
    performance_history.append(-meta_grad / len(systems))

# -----
# Test Adaptation to New Condition
# -----
test_system = AdaptiveSystem(a=1.1, b=1.0)
x = 0
target = 2.5

```

```

trajectory = []

for step in range(steps):
    u = controller.control(x, target)
    x = test_system.step(u)
    trajectory.append(x)

# -----
# Visualization
# -----

plt.figure(figsize=(8,5))
plt.plot(trajectory, label='System Output')
plt.axhline(target, color='r', linestyle='--', label='Target')
plt.xlabel('Time Step')
plt.ylabel('State')
plt.title('Adaptive Control System Output under New Condition')
plt.legend()
plt.show()

plt.figure(figsize=(8,4))
plt.plot(performance_history)
plt.xlabel('Meta-Iteration')
plt.ylabel('Average Gradient (Negative Loss)')
plt.title('Meta-Learning Performance Over Iterations')
plt.show()

```

OUTPUT:

- System output gradually converges to the target (2.5) despite new dynamics.
- Shows controller quickly adapting.

- Average gradient (negative loss) decreases over meta-iterations.
- Indicates controller gains adapt to minimize tracking error across tasks.

EXPERIMENT 5:

A team of autonomous robots collaborates to accomplish a complex task (e.g., search and rescue, package delivery) in a dynamic and uncertain environment. Implement multi-agent reinforcement learning algorithms (e.g., MADDPG, COMA) to enable the robots to coordinate their actions and achieve collective objectives efficiently. Write a Python program to simulate the robots' interactions and evaluate their performance in completing the task.

CODE:

```
import numpy as np

import matplotlib.pyplot as plt


# -----
# Environment Setup
# -----

GRID_SIZE = 5

ROBOTS = ['R1', 'R2']

TASKS = [(4,4), (0,4)] # task locations

MAX_STEPS = 20


# Initial positions

positions = {'R1': (0,0), 'R2': (4,0)}


# Action space: UP, DOWN, LEFT, RIGHT, STAY

ACTIONS = ['UP','DOWN','LEFT','RIGHT','STAY']
```



```

# -----
# Helper Functions
# -----

def move(pos, action):
    x, y = pos
    if action == 'UP': x = max(0, x-1)
    elif action == 'DOWN': x = min(GRID_SIZE-1, x+1)
    elif action == 'LEFT': y = max(0, y-1)
    elif action == 'RIGHT': y = min(GRID_SIZE-1, y+1)
    return (x,y)

def distance(p1, p2):
    return np.linalg.norm(np.array(p1)-np.array(p2))

# -----
# Simplified MADDPG Policy (Deterministic)
# -----

def actor_policy(pos, task):
    # Move towards task deterministically
    x, y = pos
    tx, ty = task
    if x < tx: return 'DOWN'
    if x > tx: return 'UP'
    if y < ty: return 'RIGHT'
    if y > ty: return 'LEFT'
    return 'STAY'

```

```

# -----
# Simulation
# -----

trajectory = {r: [positions[r]] for r in ROBOTS}
tasks_remaining = TASKS.copy()
rewards = {r:0 for r in ROBOTS}

for step in range(MAX_STEPS):
    for r in ROBOTS:
        if tasks_remaining:
            # Assign closest task
            task = min(tasks_remaining, key=lambda t: distance(positions[r], t))
            action = actor_policy(positions[r], task)
            positions[r] = move(positions[r], action)
            trajectory[r].append(positions[r])
            # Reward for completing task
            if positions[r] == task:
                rewards[r] += 10
                tasks_remaining.remove(task)

# -----
# Visualization
# -----

plt.figure(figsize=(6,6))
colors = ['blue','green']
for

```

OUTPUT:

Tasks Completed by each Robot:

R1: 1 tasks

R2: 1 tasks